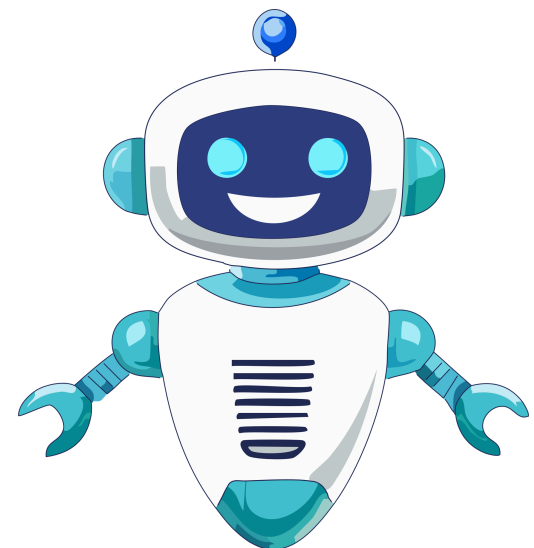


Technical Documentation

RAG System

Team 10
By Chaima MANSOURI



Medical RAG Assistant using Gemin API

System Overview :

This project implements a Retrieval-Augmented Generation (RAG) system that acts as a Medical AI Assistant. The system answers user questions by retrieving relevant medical information from a dataset and generating responses using the Gemini API.

The workflow is:

User Question → Retrieval from Vector Database → Context Creation → Gemini Response Generation.

Agent Persona :

The agent follows the Personal Assistant persona and is designed to help users by:

- Answering questions about patients
- Explaining medical reports
- Providing information about diseases and medications

DataSet :

The system uses a synthetic medical dataset consisting of: patients.json containing structured patient data (ID, age, disease, medication, etc.).

- PDF medical reports containing laboratory results such as blood sugar, hemoglobin, and cholesterol.
- Text from these sources is extracted and used as the knowledge base.

Embeddings :

Text data is converted into vector embeddings using the SentenceTransformer model (all-MiniLM-L6-v2).

These embeddings allow the system to perform semantic similarity searches between user questions and stored documents.

Vector Database :

The project uses FAISS as the vector database to store embeddings and perform efficient similarity search.

When a user asks a question, the system retrieves the top 15 most similar document chunks.

Retrieval and Re-ranking

The retrieval process occurs in two steps:

Vector Search (FAISS) to find similar documents.

Re-ranking using CrossEncoder (ms-marco-MiniLM-L-6-v2) to improve accuracy.

The system selects the top 5 most relevant results to build the context.

Prompting Strategy

A structured prompt is sent to the language model containing:

- Instructions for the assistant
- The retrieved medical context
- The user's question
- The model is instructed to answer only using the provided context.

Language Model

The system uses the Gemini API with the model **gemini-3-flash-preview** to generate responses based on the retrieved context.

User Interface

The interface is built using Streamlit, providing a chat-based interaction where users can ask questions and receive answers in real time.

Intent Detection, Chat Memory

- The system detects simple user intents such as greetings, goodbyes, and thanks. If the message is small talk, the system replies with predefined responses without running the full RAG pipeline.
- The application keeps conversation history using Streamlit session state (`st.session_state.messages`), allowing previous messages to remain visible during the session.

Technologies Used

- Python
- Streamlit
- FAISS
- SentenceTransformers
- CrossEncoder
- pypdf
- Gemini API
- NumPy

Future Improvements

- Possible improvements include:
- Hybrid search (BM25 + embeddings)
- Better document chunking
- Larger datasets
- Multi-turn conversation support